LABSHEET - 3

Q1. Hello Script

Create hello.sh using a shebang and a variable that prints a personalized greeting. Make it executable and run it using ./hello.sh and bash hello.sh.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cd ~/linux_lab3
echo '#!/bin/bash
name="Student"
echo "Hello, $name!"' > hello.sh
chmod +x hello.sh
./hello.sh
bash hello.sh
```

Q2. User Input Script

Create user_input.sh that asks for a student's name, branch and semester using read, then prints a formatted welcome message.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > user_input.sh
#!/bin/bash
echo -n "Enter student's name: "
read name
echo -n "Enter branch: "
read branch
echo -n "Enter semester: "
read sem
echo "Welcome, $name from $branch, Semester $sem!"
EOF
chmod +x user_input.sh
./user_input.sh
```

Q3. Arithmetic Calculator

Create calculator.sh that reads two integers and displays their sum, difference, product, and quotient using Bash arithmetic.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > calculator.sh
#!/bin/bash
echo -n "Enter first integer: "
```

```
read a
echo -n "Enter second integer: "
read b
sum=$((a + b))
diff=$((a - b))
prod=$((a * b))
if [ $b -ne 0 ]; then
   quot=$((a / b))
else
   quot="Undefined (division by zero)"
fi
echo "Sum: $sum"
echo "Difference: $diff"
echo "Product: $prod"
echo "Quotient: $quot"
EOF
chmod +x calculator.sh
./calculator.sh
```

Q4. Command Substitution

Create system_info.sh that stores current date, working directory, and logged-in username in variables using command substitution.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > system_info.sh
#!/bin/bash
cur_date=$(date)
cur_dir=$(pwd)
username=$(whoami)
echo "Current Date and Time: $cur_date"
echo "Current Working Directory: $cur_dir"
echo "Logged-in Username: $username"
EOF
chmod +x system_info.sh
./system_info.sh
```

Q5. Number Classification

Create number_type.sh that reads an integer and reports whether it is positive, negative, or zero using if/elif/else.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > number_type.sh
#!/bin/bash
echo -n "Enter an integer: "
```

```
read num
if [ $num -gt 0 ]; then
   echo "$num is Positive."
elif [ $num -lt 0 ]; then
   echo "$num is Negative."
else
   echo "The number is Zero."
fi
EOF
chmod +x number_type.sh
./number_type.sh
```

Q6. Even or Odd

Create even_odd.sh that accepts one integer as $1 and reports whether it is even or odd. If argument is missing, exit with non-zero status.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > even_odd.sh
#!/bin/bash
if [ -z "$1" ]; then
   echo "Usage: $0 <integer>"
   exit 1
fi
if [ $(( $1 % 2 )) -eq 0 ]; then
   echo "$1 is Even."
else
   echo "$1 is Odd."
fi
EOF
chmod +x even_odd.sh
./even_odd.sh 8
```

Q7. File and Directory Tests

Create file_check.sh that accepts a path as $1 and reports whether it is a regular file, directory, or neither, and if executable.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > file_check.sh
#!/bin/bash
if [ -z "$1" ]; then
   echo "Usage: $0 <path>"
   exit 1
fi
```

```
if [ -d "$1" ]; then
   echo "$1 is a Directory."
elif [ -f "$1" ]; then
   echo "$1 is a Regular File."
   if [ -x "$1" ]; then
      echo "It is executable."
   else
      echo "It is not executable."
   fi
else
   echo "$1 is neither a regular file nor a directory."
fi
EOF
chmod +x file_check.sh
./file_check.sh data
```

Q8. For Loop Practice

Create multiplication.sh that accepts a number as $1 and prints its multiplication table from 1 to 10 using a for loop.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > multiplication.sh
#!/bin/bash
if [ -z "$1" ]; then
   echo "Usage: $0 <number>"
   exit 1
fi
for i in {1..10}; do
   echo "$1 x $i = $(( $1 * i ))"
done
EOF
chmod +x multiplication.sh
./multiplication.sh 5
```

Q9. Loop Through Files

Create list_txt.sh that loops through all .txt files inside the data directory and prints each filename.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > list_txt.sh
#!/bin/bash
for file in data/*.txt; do
   echo "Found text file: $file"
done
```

```
EOF
chmod +x list_txt.sh
./list_txt.sh
```

Q10. While Loop

Create countdown.sh that uses a while loop to print a countdown from 10 to 1 and then prints 'Done!'.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > countdown.sh
#!/bin/bash
i=10
while [ $i -ge 1 ]; do
   echo $i
   i=$((i - 1))
done
echo "Done!"
EOF
chmod +x countdown.sh
./countdown.sh
```

Q11. Script Arguments

Create arguments.sh that displays $0, $1, $2, $#, and $@. Test it with arguments.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > arguments.sh
#!/bin/bash
echo "Script Name (\$0): $0"
echo "First Argument (\$1): $1"
echo "Second Argument (\$2): $2"
echo "Total Arguments (\$#): $#"
echo "All Arguments (\$@): $@"
EOF
chmod +x arguments.sh
./arguments.sh Hello World
```

Q12. Exit Codes

Create status.sh that accepts a directory name. Exit with 0 if it exists, exit with 1 if not. Inspect with echo $?.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > status.sh
#!/bin/bash
if [ -d "$1" ]; then
   exit 0
else
   exit 1
fi
EOF
chmod +x status.sh
./status.sh data
echo $?
```

Q13. Output Redirection

Write commands that redirect data directory listing to output/files.txt (overwriting > then appending >>).

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
ls data > output/files.txt
ls backups >> output/files.txt
cat output/files.txt
```

Q14. Append a Log

Create log_entry.sh that appends timestamp and user message to logs/activity.log without overwriting.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > log_entry.sh
#!/bin/bash
echo -n "Enter log message: "
read msg
echo "$(date): $msg" >> logs/activity.log
echo "Log entry saved."
EOF
chmod +x log_entry.sh
./log_entry.sh
```

Q15. Input Redirection and Sorting

Use data/names.txt and input redirection/pipes to sort names and remove duplicates.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
sort < data/names.txt | uniq
```

Q16. Error Logging

Run a command producing an error, redirect standard error to logs/errors.log, and display the log.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
ls non_existent_directory 2> logs/errors.log
cat logs/errors.log
```

Q17. Combined Output and Error Logging

Redirect standard output and standard error of a command into logs/combined.log and verify.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
ls data non_existent_directory > logs/combined.log 2>&1
cat logs/combined.log
```

Q18. Pipeline Practice

Use pipeline with ps aux and grep to find matching processes, then count lines.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
ps aux | grep bash
ps aux | grep bash | wc -l
```

Q19. Backup Checker Script

Create backup_check.sh to verify directory, scan/count .bak files, and log if count is zero.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > backup_check.sh
#!/bin/bash
if [ -z "$1" ] || [ ! -d "$1" ]; then
   echo "Invalid directory."
   exit 1
fi
count=0
for file in "$1"/*.bak; do
   if [ -f "$file" ]; then
      echo "Found backup: $file"
      count=$((count + 1))
   fi
done
```

```
if [ $count -eq 0 ]; then
   echo "No backup files found" >> logs/backup_log.txt
fi
EOF
chmod +x backup_check.sh
./backup_check.sh backups
```

Q20. Integrated File Report

Create file_report.sh that accepts one filename, checks readability, displays wc counts, and logs history.

Required evidence/output: Save the script, run it successfully, and record the command(s) and output in your lab record.

```
cat << 'EOF' > file_report.sh
#!/bin/bash
if [ -z "$1" ] || [ ! -r "$1" ]; then
   echo "Error: File missing or unreadable."
   exit 1
fi
echo "--- File Report for $1 ---"
wc "$1"
echo "$(date): Processed $1 successfully" >> logs/execution_history.log
EOF
chmod +x file_report.sh
./file_report.sh data/notes.txt
```